

Chapter Overview

Chapter 17 covers Kubernetes (K8s) for ShopVerse: core objects (Pod, Deployment, Service, ConfigMap, Secret), scheduling, HPA, rolling updates, PVC, RBAC, NetworkPolicy, Helm, GitOps with ArgoCD, resource management, and production readiness patterns.

Sub-Chapter	Topic
17-01	Kubernetes Architecture – control plane, worker nodes, etcd
17-02	Pod – smallest deployable unit, lifecycle, init containers
17-03	Deployment – ReplicaSet, rolling update, rollback
17-04	Service – ClusterIP, NodePort, LoadBalancer, headless
17-05	ConfigMap & Secret – externalise configuration
17-06	HPA – Horizontal Pod Autoscaler, custom metrics
17-07	PVC & StorageClass – persistent storage for stateful apps
17-08	RBAC – Role, ClusterRole, RoleBinding, service accounts
17-09	NetworkPolicy – pod-to-pod traffic control
17-10	Helm – chart templating, values, releases
17-11	GitOps with ArgoCD – declarative continuous delivery
17-12	Production Readiness – PodDisruptionBudget, topology spread, OPA

Kubernetes Cluster

Control Plane (master nodes – 3 for HA)

kube-apiserver – all requests via REST; validates & persists to etcd

etcd – distributed key-value store; all cluster state

kube-scheduler – assigns Pods to nodes (resource fit, affinity, taints)

kube-controller-manager – Deployment, ReplicaSet, HPA controllers

Worker Nodes (data plane – 3+ nodes)

kubelet – ensures Pods run as specified in PodSpec

kube-proxy – iptables/ipvs rules for Service routing

Container Runtime – containerd (OCI); runs actual containers

CNI Plugin – Flannel/Calico for pod networking

Component	Role	Failure Impact
kube-apiserver	All client and component requests go through it	Cluster unmanageable; existing pods keep running
etcd	Persists all objects (Pods, Deployments, Secrets)	Total cluster loss if all etcd nodes fail
kube-scheduler	Places new Pods on nodes	New Pods stay Pending; existing pods unaffected
kubelet	Keeps containers running per PodSpec	Pods on that node not restarted if they crash

```

apiVersion: v1
kind: Pod
metadata: { name: product-pod, namespace: shopverse }
spec:
  initContainers:
  - name: wait-for-db
    image: busybox:1.36
    command: ["sh", "-c",
      "until nc -z postgres 5432; do echo waiting; sleep 2; done"]
  containers:
  - name: product-service
    image: ghcr.io/shopverse/product-service:abc1234
    ports: [{ containerPort: 8080 }]
    envFrom: [{ configMapRef: { name: product-config } }]
    env: [{ name: DB_PASSWORD, valueFrom: { secretKeyRef:
      { name: db-secret, key: password } } }]
  resources:
    requests: { memory: "512Mi", cpu: "250m" }
    limits: { memory: "1Gi", cpu: "1000m" }

```

Pod Phase	Meaning	Next State
Pending	Scheduled; images being pulled; init containers running	Running
Running	At least one container running	Succeeded / Failed
Succeeded	All containers exited 0; restartPolicy=Never	Terminated
Failed	At least one container exited non-zero; Restart per policy or terminal	Restart
Unknown	Node communication lost	May recover or be garbage collected

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: product-service, namespace: shopverse }
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 0 # never reduce below 3 pods during update
    maxSurge: 1 # allow 1 extra pod during update (4 total briefly)
  selector: { matchLabels: { app: product-service } }
  template:
    metadata: { labels: { app: product-service } }
    spec:
      containers:
      - name: product-service
        image: ghcr.io/shopverse/product-service:abc1234
        readinessProbe:
          httpGet: { path: /actuator/health/readiness, port: 8080 }
          initialDelaySeconds: 30 failureThreshold: 3
        livenessProbe:
          httpGet: { path: /actuator/health/liveness, port: 8080 }
          initialDelaySeconds: 60 failureThreshold: 3
        # Rollback if new version fails:
        kubectrl rollout undo deployment/product-service --to-revision=2
```

Service Type	Accessible From	Use Case	ShopVerse
ClusterIP (default)	Inside cluster only	Service-to-service communication	inventory-service:8082 from order-service
NodePort	External via NodeIP:port	Development (9000, 32767) production Local traffic	External traffic
LoadBalancer	External; cloud load balancer	Production external traffic	Not used (Nginx Ingress preferred)
Headless (clusterIP:DNS)	StatefulSet turns pod IPs	StatefulSet discovery; Kafka	Kafka, Cassandra pods

```
# ClusterIP Service - internal routing
apiVersion: v1
kind: Service
metadata: { name: product-service, namespace: shopverse }
spec:
  selector: { app: product-service }
  ports: [{ port: 8080, targetPort: 8080 }]
  type: ClusterIP
# DNS: product-service.shopverse.svc.cluster.local:8080
# Ingress routes external traffic to this Service
# Nginx Ingress → product-service:8080 → Pods
```

```
# ConfigMap - non-sensitive config
apiVersion: v1
kind: ConfigMap
metadata: { name: product-config, namespace: shopverse }
data:
  SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/shopverse
  SPRING_REDIS_HOST: redis
  SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092
  MANAGEMENT_ENDPOINTS_WEB_EXPOSURE_INCLUDE: health,metrics,prometheus
---
# Secret - sensitive values (base64-encoded)
apiVersion: v1
kind: Secret
metadata: { name: db-secret, namespace: shopverse }
type: Opaque
stringData:
  password: shopverse_prod_pass # stringData: plain text; K8s encodes
```

Method	Use	Limitations
envFrom: configMapRef	All CM keys as env vars	Env vars not updated until pod restart
env: valueFrom.secretKeyRef	Single secret key as env var	Same; restart to pick up changes
Volume mount (ConfigMap)	Mount as file; dynamic reload possible	More complex; app must watch file
External Secrets Operator	Sync AWS Secrets Manager → K8s Secret	Best Secret; rotation support

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: product-service-hpa, namespace: shopverse }
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: product-service
  minReplicas: 3
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: { type: Utilization, averageUtilization: 70 }
  - type: Resource
    resource:
      name: memory
      target: { type: Utilization, averageUtilization: 80 }
  behavior:
    scaleUp: { stabilizationWindowSeconds: 60 }
    scaleDown: { stabilizationWindowSeconds: 300 }

```

Setting	Value	Reason
minReplicas	3	HA: survives 1 node failure + 1 pod eviction
maxReplicas	10	Cost cap; beyond 10 investigate root cause
stabilizationWindowSeconds (down)	300s	Prevents flapping: don't scale down within 5 min of scale-up

```
# StorageClass - defines how volumes are provisioned
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata: { name: fast-ssd }
provisioner: ebs.csi.aws.com
parameters: { type: gp3, iops: "3000", throughput: "125" }
reclaimPolicy: Retain
---
# PersistentVolumeClaim - request storage
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: postgres-pvc, namespace: shopverse }
spec:
  storageClassName: fast-ssd
  accessModes: [ReadWriteOnce]
  resources: { requests: { storage: 100Gi } }
```

Access Mode	Abbreviation	Use Case
ReadWriteOnce	RWO	One node reads+writes; most block storage (EBS, GCE PD)
ReadOnlyMany	ROX	Many nodes read; shared static assets
ReadWriteMany	RWX	Many nodes read+write; NFS, EFS, Ceph

Reclaim Policy	After PVC Delete	Use For
Retain	PV stays; manual cleanup	Production – don't lose data on accidental PVC delete
Delete	PV and volume deleted	Dev/test; ephemeral data


```
# ServiceAccount for product-service
apiVersion: v1
kind: ServiceAccount
metadata: { name: product-sa, namespace: shopverse }
---
# Role – namespace-scoped permissions
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: product-role, namespace: shopverse }
rules:
- apiGroups: [""]
  resources: ["configmaps","secrets"]
  verbs: ["get","list","watch"]
---
# RoleBinding – bind Role to ServiceAccount
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: product-rb, namespace: shopverse }
subjects:
- kind: ServiceAccount
  name: product-sa
  namespace: shopverse
roleRef: { apiGroup: rbac.authorization.k8s.io, kind: Role, name: product-role }
```

Object	Scope	Use Case
Role	Namespace	Grant access to namespace resources (pods, configmaps, secrets)
ClusterRole	Cluster-wide	Grant access to cluster resources (nodes, PVs, CRDs)
RoleBinding	Namespace	Bind Role or ClusterRole to subject in one namespace
ClusterRoleBinding	Cluster-wide	Bind ClusterRole to subject across all namespaces

```
# Default deny all ingress in shopverse namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny-all, namespace: shopverse }
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
---
# Allow product-service to receive from api-gateway only
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-gateway-to-product, namespace: shopverse }
spec:
  podSelector: { matchLabels: { app: product-service } }
  ingress:
  - from:
    - podSelector: { matchLabels: { app: api-gateway } }
  ports: [{ protocol: TCP, port: 8080 }]
```

Policy	Effect
Default deny all (empty podSelector and policyTypes)	Deny all traffic without an explicit allow rule
Allow gateway → product on 8080	product-service only accepts from api-gateway
Allow product → postgres on 5432	product-service can reach postgres; nothing else can
Allow product → redis on 6379	product-service can reach redis; isolated from other pods

```
# Helm chart structure
shopverse-chart/
Chart.yaml # name, version, description
values.yaml # default values
values-prod.yaml # production overrides
templates/
deployment.yaml
service.yaml
hpa.yaml
configmap.yaml
# templates/deployment.yaml snippet
image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
replicas: {{ .Values.replicaCount }}
resources: {{- toYaml .Values.resources | nindent 12 }}
# values.yaml
replicaCount: 3
image:
repository: ghcr.io/shopverse/product-service
tag: latest
# Deployment commands
helm install product-svc ./shopverse-chart -f values-prod.yaml -n shopverse
helm upgrade product-svc ./shopverse-chart --set image.tag=abc1234 -n shopverse
helm rollback product-svc 2 -n shopverse
```

GitOps Flow

Git Repository (shopverse-k8s-config) – single source of truth

developer pushes Helm chart / K8s manifests to Git

ArgoCD detects diff between Git state and cluster state

Polls Git every 3 minutes OR webhook push trigger

ArgoCD syncs cluster to match Git (auto or manual sync)

kubectl apply of changed manifests to shopverse namespace

Deployment rolls out; ArgoCD reports Healthy/Progressing/Degraded

Rollback: revert commit in Git → ArgoCD syncs back to old state

```
# ArgoCD Application definition
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: product-service, namespace: argocd }
spec:
  project: default
  source:
    repoURL: https://github.com/shopverse/k8s-config
    targetRevision: main
  path: apps/product-service
  destination: { server: https://kubernetes.default.svc, namespace: shopverse }
  syncPolicy:
    automated: { prune: true, selfHeal: true }
```

PodDisruptionBudget – survive voluntary disruptions

```

apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: product-pdb, namespace: shopverse }
spec:
  maxUnavailable: 1
  selector: { matchLabels: { app: product-service } }
# K8s node drain respects PDB: will not evict if it would violate maxUnavailable

```

Topology Spread Constraints – spread pods across zones

```

topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
  labelSelector: { matchLabels: { app: product-service } }

```

Production Concern	K8s Solution
Pod HA during node maintenance	PodDisruptionBudget (maxUnavailable: 1)
Even spread across AZs	topologySpreadConstraints by zone
Resource starvation	resources.requests and limits on every container
Secret rotation	External Secrets Operator → auto-sync from AWS Secrets Manager
Policy enforcement (OPA)	Gatekeeper ConstraintTemplate; enforce imagePullPolicy, non-root

■ Do

- ✓ Set resources.requests AND limits on every container – scheduler needs requests; limits prevent starvation.
- ✓ Use PodDisruptionBudget (maxUnavailable:1) to ensure HA during node drains.
- ✓ Set readiness and liveness probes – K8s only routes traffic to ready pods; restarts unhealthy ones.
- ✓ Use topologySpreadConstraints to spread pods across availability zones.
- ✓ Use Helm for K8s manifests – templated, versioned, rollback support.
- ✓ Use GitOps (ArgoCD) – Git is the single source of truth; no kubectl apply in CI.
- ✓ Use namespace-scoped Roles over ClusterRoles – principle of least privilege.
- ✓ Apply NetworkPolicy default-deny; add explicit allow rules – zero-trust pod networking.
- ✓ Tag Docker images with git SHA in Deployments – immutable; no surprise on imagePull.
- ✓ Use External Secrets Operator for production secrets – rotation without pod restart.

■ Anti-Patterns

- No resource requests/limits – unbounded pod starves other pods on same node.
- No readiness probe – K8s sends traffic to starting pod; 500 errors during rollout.
- Running all pods on same node – no HA; one node failure kills all replicas.
- Storing secrets in ConfigMaps (plain text) – use Secrets (base64) or External Secrets.
- Using kubectl apply directly in CI (imperative) – drifts from Git state; no audit trail.
- Using latest tag in Deployment – two pods may pull different images on restart.
- ClusterRole for namespace-scoped needs – violates least privilege; use Role in namespace.

Sub-Chapter	Key Point	One-Liner
17-01 Architecture	Control plane + worker nodes	apiserver → etcd; kubelet on workers
17-02 Pod	Init containers	wait-for-db init before app container starts
17-03 Deployment	maxUnavailable:0, maxSurge:1	Zero-downtime rolling update; readiness probe gates
17-04 Service	ClusterIP + Ingress	DNS: svc.ns.svc.cluster.local; Nginx routes external
17-05 ConfigMap/Secret	Map/Secret secretKeyRef	External Secrets Operator for production
17-06 HPA	CPU 70%; min 3 max 10	stabilizationWindowSeconds:300 prevents flapping
17-07 PVC	StorageClass fast-ssd	ReclaimPolicy:Retain for prod; RWO for block storage
17-08 RBAC	Role + RoleBinding + SA	Namespace-scoped; least privilege
17-09 NetworkPolicy	Default deny all	Explicit allow gateway→product→postgres→redis
17-10 Helm	values.yaml + templates	helm upgrade --set image.tag=sha; helm rollback
17-11 GitOps	ArgoCD selfHeal:true	Git commit → auto sync → cluster matches
17-12 Prod Ready	PDB + topology spread	maxUnavailable:1; spread by AZ zone key

Interview Quick-Check

Question	Concise Answer
Liveness vs Readiness probe?	Liveness: is JVM alive? Failure → restart pod. Readiness: is app ready for traffic? Failure → remove from service
HPA scaling behaviour?	HPA reads CPU/memory from metrics-server. Scales up quickly (60s window). Scale-down stabilization (300s) prevents flapping
Why PodDisruptionBudget?	During voluntary disruptions (node drain, upgrade), K8s respects PDB.maxUnavailable. Ensures at least 1 replica available
Helm vs raw YAML?	Helm: parameterised templates; release management; rollback. Raw YAML: simpler; no templating. Helm charts are reusable
GitOps benefit?	Git = source of truth. ArgoCD detects cluster drift and auto-heals. Every change audited via Git commit history